Rapport d'Analyse de Bugs

Projet IHM_Control — Raspberry Pi 4

Auteur :	Mohamed Selmani
LinkedIn :	linkedin.com/in/mohamed-selmani
Projet :	IHM_Control (GitHub : selmanithe1/IHM_Control)
Date :	Mars 2026
Version :	1.0
Fichiers :	motor_control.py, encoder.py, ui_control.py, endurance_cycles.py, joystick_control.py

Résumé exécutif

Ce rapport identifie **8 bugs critiques** dans le système IHM_Control. Chaque bug est documenté avec son code source réel, un exemple concret d'impact, et une solution proposée.

Table des matières

1 Tableau Récapitulatif des Bugs **2**

2 Bug #1 — Double Incrémentation de `homing_count_a` **3**
2.1 Description et Code Concerné . . . 3
2.2 Exemple Réel Concret . . . 3
2.3 Solution . . . 3

3 Bug #2 — `initialize_position_b()` Jamais Exécutée si A Échoue **5**
3.1 Description et Code Concerné . . . 5
3.2 Exemple Réel Concret . . . 5
3.3 Solution . . . 5

4 Bug #3 — `except: pass` Avale les Erreurs Silencieusement **6**
4.1 Description et Code Concerné . . . 6
4.2 Exemple Réel Concret . . . 6
4.3 Solution . . . 6

5 Bug #4 — Variable `deadline` Non Définie dans `_wait_for_stall` **7**
5.1 Description et Code Concerné . . . 7
5.2 Exemple Réel Concret . . . 7
5.3 Solution . . . 7

6 Bug #5 — Import Circulaire Latent entre `endurance_cycles` et `ui_control_2w` **8**
6.1 Description et Code Concerné . . . 8
6.2 Exemple Réel Concret . . . 8
6.3 Solution . . . 8

7 Bug #6 — Race Condition : Joystick vs Endurance 4W **9**
7.1 Description et Code Concerné . . . 9
7.2 Exemple Réel Concret . . . 9
7.3 Solution . . . 9

8 Bug #7 — Popup Toast Fermée Après 2s Mais Init Dure 15–20s **10**
8.1 Description et Code Concerné . . . 10
8.2 Exemple Réel Concret . . . 10
8.3 Solution . . . 10

9 Bug #8 — `pigpio.pi()` au Niveau Module dans `encoder.py` **11**
9.1 Description et Code Concerné . . . 11
9.2 Exemple Réel Concret . . . 11
9.3 Solution . . . 11

10 Conclusion et Plan d'Action **13**

1. Tableau Récapitulatif des Bugs

N°	Fichier	Description	Sévérité
1	motor_control.py	Double incrémentation de `homing_count_a`	Haute
2	ui_control.py	`initialize_position_b()` jamais appelée si A échoue	Critique
3	motor_control.py	`except: pass` avale silencieusement les erreurs	Haute
4	motor_control.py	Variable `deadline` potentiellement non définie	Critique
5	endurance_cycles.py	Import circulaire latent	Moyenne
6	joystick_control.py	Race condition joystick vs endurance 4W	Haute
7	ui_control.py	Popup fermée après 2s mais init dure 15–20s	Haute
8	encoder.py	`pigpio.pi()` au niveau module crash au démarrage	Critique

2. Bug #1 — Double Incrémentation de homing_count_a

2.1. Description et Code Concerné

Bug #1 — motor_control.py, lignes 334 et 344

La variable `homing_count_a` est incrémentée deux fois lors d'un seul cycle d'initialisation : une fois dans `initialize_position_a()` et une seconde fois dans la sous-fonction `_do_homing_a()`. Résultat : le compteur double inutilement.

Listing 1 – Code original — motor_control.py lignes 330-350

```
def initialize_position_a():
    global homing_count_a
    ...
    homing_count_a += 1    # LIGNE 334: premier increment
    ...
    _do_homing_a()         # appel interne
    ...

def _do_homing_a():
    global homing_count_a
    ...
    homing_count_a += 1    # LIGNE 344: DOUBLE increment!
    ...
```

2.2. Exemple Réel Concret

Exemple Concret

Scénario : L'utilisateur appuie sur le bouton **Initialisation** en mode 4W.
Attendu : `homing_count_a` passe de 0 à 1 après un cycle.
Résultat observé : `homing_count_a` passe de 0 à 2 après un seul appui.
Après 5 appuis : compteur = 10 au lieu de 5. Toute logique conditionnelle basee sur ce compteur (ex : `if homing_count_a > 3`) se déclenche trop tôt et peut forcer une ré-initialisation non souhaitée.

2.3. Solution

Solution — Supprimer le double increment

Garder l'incrémentation uniquement dans `initialize_position_a()` et supprimer celle dans _do_homing_a().

Listing 2 – Code corrigé

```
def initialize_position_a():
    global homing_count_a
    ...
    homing_count_a += 1    # Un seul endroit
    ...
```

```
    _do_homing_a()         # plus d'increment ici

def _do_homing_a():
    # global homing_count_a SUPPRIME
    # homing_count_a += 1  SUPPRIME
    ...
```

3. Bug #2 — initialize_position_b() Jamais Exécutée si A Échoue

3.1. Description et Code Concerné

Bug #2 — ui_control.py, lignes 1529–1531

En mode 4W, la séquence d'initialisation appelle d'abord `initialize_position_a()` puis `initialize_position_b()`. Cependant, si la première échoue ou lève une exception, la deuxième n'est **jamais appelée**.

Listing 3 – Code original — ui_control.py lignes 1527-1535

```
def _run_init_4w():
    try:
        initialize_position_a()  # Si ceci echoue...
        initialize_position_b()  # ...ceci ne s'execute JAMAIS
    except Exception as e:
        log_message(f'Init error: {e}')
    # Le moteur B reste non initialise !
```

3.2. Exemple Réel Concret

Exemple Concret

Scénario : Le moteur A rencontre une obstruction mécanique pendant l'homing. `initialize_position_a()` lève `RuntimeError`.
Résultat : `initialize_position_b()` n'est pas appelée. Le moteur B reste avec `limits_set_b = False`.
Quand l'utilisateur déplace ensuite le moteur B via joystick, le système croit que les limites ne sont pas définies et n'applique aucune protection contre les dépassements de course — risque de détérioration mécanique !

3.3. Solution

Solution — Blocs try/except séparés

Sacrifie l'atomicité pour garantir que chaque moteur est initialisé indépendamment même si l'autre échoue.

Listing 4 – Code corrigé

```
def _run_init_4w():
    try:
        initialize_position_a()
    except Exception as e:
        log_message(f'Init A error: {e}')
    try:
        initialize_position_b()  # Toujours execute
    except Exception as e:
        log_message(f'Init B error: {e}')
```

4. Bug #3 — `except: pass` Avale les Erreurs Silencieusement

4.1. Description et Code Concerné

Bug #3 — motor_control.py, ligne 87

L'utilisation de `except: pass` sans spécifier le type d'exception ni journaliser l'erreur rend le débogage impossible. Toute exception est silencieusement ignorée.

Listing 5 – Code original — motor_control.py ligne 85-90

```
def save_state():
    try:
        with open(STATE_FILE, 'w') as f:
            json.dump(state_data, f)
    except:    # LIGNE 87: capture TOUT, log RIEN
        pass   # L'erreur disparait sans trace !
```

4.2. Exemple Réel Concret

Exemple Concret

Scénario : Le système de fichiers est en lecture seule (disque plein ou corruption). `save_state()` échoue silencieusement.
Résultat : L'état du moteur (position, limites) n'est jamais sauvegardé. Au redémarrage, le robot pense être à la position 0 alors qu'il est physiquement à la position 500. Risque de collision mécanique immédiate !
L'utilisateur ne voit aucun message d'erreur et croit que tout va bien.

4.3. Solution

Listing 6 – Code corrigé

```
def save_state():
    try:
        with open(STATE_FILE, 'w') as f:
            json.dump(state_data, f)
    except PermissionError as e:
        log_message(f'[ERREUR] Sauvegarde impossible: {e}')
    except OSError as e:
        log_message(f'[ERREUR] Erreur systeme: {e}')
```

5. Bug #4 — Variable `deadline` Non Définie dans `_wait_for_stall`

5.1. Description et Code Concerné

Bug #4 — motor_control.py, lignes 204–218

Dans la fonction `_wait_for_stall()`, la variable `deadline` est utilisée dans la condition `while` mais elle peut ne pas être initialisée si l'exécution atteint la boucle sans passer par le code d'initialisation.

Listing 7 – Code original — motor_control.py lignes 204-218

```
def _wait_for_stall(read_pos_func, timeout_s=HOMING_TIMEOUT,
                    sample_dt=HOMING_DT, still_threshold=HOMING_STILL):
    last = read_pos_func()
    # deadline n'est PAS initialisee ici dans certains chemins
    while time.time() < deadline:  # NameError possible!
        time.sleep(sample_dt)
        cur = read_pos_func()
        if abs(cur - last) <= still_threshold:
            return cur
        last = cur
    return read_pos_func()
```

5.2. Exemple Réel Concret

Exemple Concret

Scénario : L'homing est déclenché dans un contexte inattendu.
Résultat potentiel : Python lève `NameError: name 'deadline' is not defined`. L'initialisation s'arrête brusquement, les moteurs restent en mouvement, le flag `limits_set_a` reste à `False`, le joystick reste désactivé.

5.3. Solution

Listing 8 – Code corrigé

```
def _wait_for_stall(read_pos_func, timeout_s=HOMING_TIMEOUT,
                    sample_dt=HOMING_DT, still_threshold=HOMING_STILL):
    last = read_pos_func()
    deadline = time.time() + timeout_s  # TOUJOURS initialiser ici
    while time.time() < deadline:
        time.sleep(sample_dt)
        cur = read_pos_func()
        if abs(cur - last) <= still_threshold:
            return cur
        last = cur
    return read_pos_func()
```

6. Bug #5 — Import Circulaire Latent entre endurance_cycles et ui_control_2w

6.1. Description et Code Concerné

Bug #5 — endurance_cycles.py

La fonction `_get_log_fn()` utilise un import tardif (lazy import) de `ui_control_2w.endurance_log_2w` pour contourner un import circulaire. Cependant, si cette fonction est appelée avant que l'application Kivy soit démarrée, le log retourne `None` silencieusement.

Listing 9 – Code original — endurance_cycles.py

```python
_endurance_log_2w_fn = None

def _get_log_fn():
    global _endurance_log_2w_fn
    if _endurance_log_2w_fn is None:
        try:
            from ui_control_2w import endurance_log_2w  # import tardif
            _endurance_log_2w_fn = endurance_log_2w
        except ImportError:
            pass  # Silencieux si UI non disponible
    return _endurance_log_2w_fn

def log_endurance(msg):
    fn = _get_log_fn()
    if fn:  # Si None, le message est perdu !
        fn(msg)
```

6.2. Exemple Réel Concret

Exemple Concret

Scénario : L'utilisateur lance un test d'endurance avant que l'interface Kivy soit complètement initialisée.
Résultat : Tous les messages de log pendant les premières secondes disparaissent. L'utilisateur ne voit aucune confirmation du début du cycle.

6.3. Solution

Listing 10 – Code corrigé

```python
def log_endurance(msg):
    fn = _get_log_fn()
    if fn:
        fn(msg)
    else:
        print(f'[ENDURANCE LOG - UI not ready]: {msg}')  # Fallback console
```

7. Bug #6 — Race Condition : Joystick vs Endurance 4W

7.1. Description et Code Concerné

Bug #6 — joystick_control.py / endurance_cycles.py

Lorsque le mode endurance 4W est lancé, le flag `control_flags.joystick_enabled` n'est pas mis à `False`. Les threads joystick continuent de lire les analogiques et d'envoyer des commandes moteur en parallèle du cycle d'endurance.

Listing 11 – Code original — endurance_cycles.py (probleme)

```
def launch_endurance_from_gui(cycles, speed, ...):
    # control_flags.joystick_enabled N'EST PAS desactive ici
    endurance_thread = threading.Thread(
        target=_run_endurance_4w, args=(cycles, speed, ...)
    )
    endurance_thread.start()
    # Pendant ce temps, joystick_loop() continue dans son thread !
```

Listing 12 – Code original — joystick_control.py (thread concurrent)

```
def joystick_loop():
    while running:
        if control_flags.joystick_enabled:  # Toujours True !
            val_left = read_ads1115(channel=0)
            val_right = read_ads1115(channel=1)
            set_motor_speed(val_left, val_right)  # Ecrase les
                commandes endurance!
        time.sleep(0.05)
```

7.2. Exemple Réel Concret

Exemple Concret

Scénario : Le cycle d'endurance commande le moteur A à la position 800. Simultanément, le joystick gauche est à 50% (valeur neutre avec léger bruit).
Résultat : `set_motor_speed()` depuis le thread joystick écrase la commande d'endurance toutes les 50ms. Le moteur oscille entre 800 et 600, créant des à-coups et invalidant les mesures d'endurance.

7.3. Solution

Listing 13 – Code corrigé

```
def launch_endurance_from_gui(cycles, speed, ...):
    control_flags.joystick_enabled = False  # Desactiver AVANT
    try:
        _run_endurance_4w(cycles, speed, ...)
    finally:
        control_flags.joystick_enabled = True  # Reactiver apres
```

8. Bug #7 — Popup Toast Fermée Après 2s Mais Init Dure 15–20s

8.1. Description et Code Concerné

Bug #7 — ui_control.py, lignes 1007–1009

La popup « Initialisation en cours... » est programmée pour se fermer après 2 secondes via `Clock.schedule_once`. Or, le cycle d'initialisation complette (double homing A+B) dure entre 15 et 20 secondes.

Listing 14 – Code original — ui_control.py lignes 1005-1012

```
def show_init_popup():
    toast = Popup(title='Initialisation',
                  content=Label(text='Initialisation en cours...'),
                  auto_dismiss=False)
    toast.open()
    # Fermeture apres 2s alors que l'init dure 15-20s !
    Clock.schedule_once(lambda *_: toast.dismiss(), 2.0)
    threading.Thread(target=_run_init_4w).start()
```

8.2. Exemple Réel Concret

Exemple Concret

Scénario : L'utilisateur appuie sur **Initialisation**.
Ce qui se passe :

1. t=0s : Popup « Initialisation en cours... » apparaît
2. t=2s : Popup se ferme — l'utilisateur croit que c'est fini
3. t=2s–15s : L'initialisation continue en arrière-plan (invisible)
4. L'utilisateur rappuie sur Initialisation à t=3s
5. Deux cycles d'init s'exécutent en parallèle — crash garanti !

8.3. Solution

Listing 15 – Code corrigé

```
def show_init_popup():
    toast = Popup(title='Initialisation',
                  content=Label(text='Initialisation en cours...'),
                  auto_dismiss=False)
    toast.open()

    def _init_and_close():
        _run_init_4w()  # Attendre la fin
        Clock.schedule_once(lambda *_: toast.dismiss(), 0)  # Fermer
            apres

    threading.Thread(target=_init_and_close).start()
```

9. Bug #8 — `pigpio.pi()` au Niveau Module dans `encoder.py`

9.1. Description et Code Concerné

Bug #8 — encoder.py, ligne 8

L'instruction `pi = pigpio.pi()` s'exécute au moment de l'**import** du module, pas dans une fonction. Si le daemon `pigpiod` n'est pas lancé (typiquement en développement ou en cas d'oubli), cela lève une exception et **crashe toute l'application** avant même l'affichage de l'interface.

Listing 16 – Code original — encoder.py lignes 1-12

```
1  import pigpio
2  import threading
3
4  # LIGNE 8: s'execute AU MOMENT DE L'IMPORT !
5  pi = pigpio.pi()  # Si pigpiod n'est pas lance: crash immediat
6
7  ENCODER_A_PIN = 17
8  ENCODER_B_PIN = 27
9
10 def setup_encoders():
11     pi.set_mode(ENCODER_A_PIN, pigpio.INPUT)
12     # ...
```

9.2. Exemple Réel Concret

Exemple Concret

Scénario 1 : Développeur sur Ubuntu (sans Raspberry Pi). Il lance `python main.py` pour tester l'interface — crash immédiat à l'import.
Scénario 2 : Redémarrage du Raspberry Pi après une coupure. `pigpiod` n'est pas lancé automatiquement. L'application IHM crash au démarrage, aucune interface n'apparaît, aucun message utile n'est affiché.
Message d'erreur typique :

```
1 pigpio.error: failed to connect to pigpiod
```

9.3. Solution

Listing 17 – Code corrigé

```
1  import pigpio
2  import threading
3
4  pi = None  # Initialisation diff\'{e}r\'{e}e
5
6  ENCODER_A_PIN = 17
7  ENCODER_B_PIN = 27
8
```

```python
def setup_encoders():
    global pi
    try:
        pi = pigpio.pi()  # Init ici, pas au niveau module
        if not pi.connected:
            raise RuntimeError('pigpiod non disponible')
        pi.set_mode(ENCODER_A_PIN, pigpio.INPUT)
        # ...
    except Exception as e:
        print(f'[ENCODER] Erreur connexion pigpio: {e}')
        pi = None  # Graceful degradation
```

10. Conclusion et Plan d'Action

Priorités de Correction

Priorité 1 (Critiques — à corriger immédiatement) :
- Bug #2 : initialize_position_b() jamais appelée si A échoue
- Bug #4 : Variable deadline non initialisée
- Bug #8 : pigpio.pi() crash au démarrage

Priorité 2 (Hauts — à corriger sous 1 semaine) :
- Bug #1 : Double incrémentation homing_count_a
- Bug #3 : except : pass silencieux
- Bug #6 : Race condition joystick vs endurance
- Bug #7 : Popup fermée trop tôt

Priorité 3 (Moyens — à planifier) :
- Bug #5 : Import circulaire avec fallback console

Résumé des Corrections

L'application des corrections proposées dans ce rapport permettra de :
- Éliminer les risques de collision mécanique dus à une initialisation partielle
- Garantir le démarrage même sans `pigpiod` actif
- Protéger les cycles d'endurance contre les interférences joystick
- Rendre les erreurs visibles plutôt que silencieuses
- Améliorer l'expérience utilisateur lors des initialisations